

Week 3 Assignment — The Simple Guide

Multi-Agent Customer Support System · Explained in plain, easy language

🔗 Where you left off (end of Week 2)

By the end of Week 2, you had built and tested a **single AI agent** — one agent, doing one job, answering questions directly. Week 3 takes that same idea and levels it up: instead of one agent doing everything, you now build a small **team of agents** that work together, where one of them can talk to a real database.

1. What is this assignment actually asking you to do?

In plain words: **build a customer support system made of multiple small AI agents**, where each agent has one specific job, and a "router" agent decides which one should answer.

Think of it like a call center:

- You call in with a question.
- A **receptionist** (the router agent) listens and decides: "This is a billing question" or "This is a tech question."
- They transfer you to the right **department** (the specialist agent) that actually answers you.

That's the whole assignment — build that receptionist + 2 departments, connect one department to a real database, and (as a stretch goal) build one more department that lives on a totally separate computer/service.

2. The 4 Big Steps

Step	What it means in plain English
1. Set up a database	Create a place (Supabase) to store fake customer/order data, like a spreadsheet in the cloud.
2. Build the agent team	Write code for the router + billing agent + technical agent, and test that routing works.
3. Build a Returns Agent (stretch)	A 4th agent that runs as its own separate program, reachable over the network (A2A).
4. Test everything	Ask real questions and confirm the right agent + right tool answers each one.

3. What We Actually Did — Step by Step

1 Set up your computer to run Python agent code

Before writing any AI code, your computer needs the right tools installed — like installing an app before you can use it.

- Made a project folder to keep everything organized.
- Created a **virtual environment** (a clean, isolated space just for this project, so it doesn't mix with other Python projects on your computer).
- Installed **ADK** (Agent Development Kit) — Google's toolkit for building AI agents.

2 Set up Supabase (the database)

Supabase is basically a **free online spreadsheet with superpowers** — it stores your data (customers, orders, tickets) so your agents have real information to work with, instead of made-up answers.

- Created a free Supabase account and a new project.
- Made 3 tables: `customers`, `orders`, `support_tickets` — like 3 tabs in a spreadsheet.
- Filled each table with sample rows (fake customers, fake orders) using SQL `insert` commands, so there's real data to query.

3 Got an API key for the AI "brain"

An agent needs a **model** to actually think — that's the AI itself (Gemini, in this case). To use it, you need a personal key that proves the requests are coming from you.

- Got a free API key from Google AI Studio.
- Saved it safely in a special file called `.env` — this keeps secret keys out of your actual code, so you never accidentally share them.

4 Wrote the agent code

This is the actual "brain wiring." In one Python file, we defined:

- billing_agent** — knows how to answer order status, payment, and refund questions (using simple lookup functions).
- technical_agent** — knows how to answer app-crash and troubleshooting questions.
- root_agent (the router)** — has no tools of its own. Its only job is reading the customer's message and deciding which specialist should handle it.

🔗 Why does the router have no tools?

Because its job isn't to answer — it's to *decide who* answers. Keeping it simple means it can't get confused trying to do two jobs at once.

5 Tested it in the ADK Dev UI

ADK comes with a built-in website (running on your own computer) where you can chat with your agents and literally **see** which agent answered and which tool it used — like an X-ray view of your AI system.

- Ran the dev server and opened it in the browser.

- 2. Asked test questions like "What's the status of order 1?"
- 3. Watched the trace confirm: router → billing_agent → lookup_order_status → real answer.

» What's parked for later (stretch goal)

Connecting the billing agent to **real** Supabase data through something called **MCP**, and building a separate Returns Agent reachable through **A2A**. The database and its data are already set up and ready — this part just needs to be wired up when you're ready to revisit it.

4. Important Commands You Used (and why)

Command	What it does	Why you needed it
<code>python --version</code>	Shows your installed Python version	Confirms Python is installed correctly before starting
<code>python -m venv venv</code>	Creates a virtual environment folder named "venv"	Keeps this project's packages separate from everything else on your computer
<code>.\venv\Scripts\Activate.ps1</code>	Turns on ("activates") the virtual environment	Makes sure anything you install goes into this project only, not your whole system
<code>pip install google-adk</code>	Downloads and installs the Agent Development Kit	This is the toolkit used to actually build the agents
<code>pip install python-dotenv</code>	Installs a tool for reading secret keys from a file	Lets your code safely read your API key from <code>.env</code> instead of writing it directly in code
<code>adk --version</code>	Checks that ADK installed correctly	Quick sanity check before writing any agent code
<code>adk web</code>	Starts the local Dev UI website for testing agents	Lets you chat with your agents and see exactly how they think and respond
<code>python -c "import support_agent.agent"</code>	Quickly checks your code has no errors, without starting the whole server	Fast way to catch typos/mistakes before testing in the browser

Key SQL commands (used inside Supabase)

Command	What it does
<code>insert into customers (...) values (...)</code>	Adds new rows (fake customer data) into a table
<code>select * from customers;</code>	Shows every row in the customers table — used to check the data actually saved
<code>truncate table ... restart identity;</code>	Empties a table completely and resets its ID counter back to 1 — used to fix accidental duplicate data

5. Tools Used This Week (and why each one matters)

Tool	What it is	Why we used it
Python	The programming language	ADK and almost all AI agent code is written in Python
Virtual Environment (venv)	An isolated mini-computer just for this project	Stops different projects' installed packages from clashing with each other
ADK (Agent Development Kit)	Google's framework for building AI agents	Handles all the hard parts (routing, tool-calling, chat memory) so you just describe what each agent should do
Gemini API key	Your personal access pass to Google's AI model	The "brain" that actually reads messages and decides what to do — agents can't think without it
Supabase	A free online database (cloud spreadsheet)	Stores real customer/order data so agents can eventually look up real information instead of guessing
ADK Dev UI	A local test website that comes with ADK	Lets you chat with your agents and see a visual trace of exactly what happened step by step
PowerShell	Windows' command-line tool	Used to run every command above — install packages, activate environments, start servers

6. Quick Recap — Where You Stand Right Now

✔ Done

Environment set up · Supabase database created and seeded with real data · Router + billing + technical agents built and working · Routing tested and confirmed correct in the Dev UI trace

» Optional next steps (when you're ready)

Connect billing_agent to real Supabase data via MCP · Build a Returns Agent and expose it via A2A · Test all 3 final scenarios and record a demo